

Chapter 04: Dynamic Programming Algorithm

Design and Analysis of Computer Algorithms

宫秀军

天津大学智能与计算学部

Email: gongxj@tju.edu.cn

April, 2025

- 1 What is DP
 - Multi-Stage Weighted Graph
 - Definition
 - Steps
- 2 0/1 Knapsack Problem
- 3 Matrix Multiplication Chains
- 4 All Pairs Shortest Path
- 5 Maximum Non-crossing Subset of Nets
- 6 Longest Common Subsequences

Problem statement

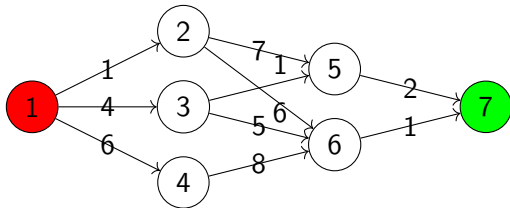
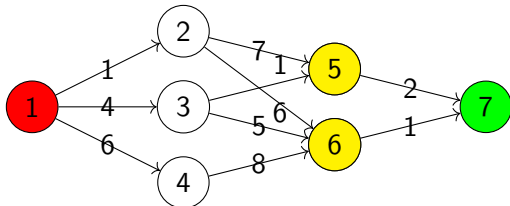


Figure 1: Multi-Stage Weighted Graph

A **MSWG** $G = (V, E)$ is a directed graph where vertices are partitioned into k (where $k > 1$) number of disjoint subsets $S = \{S_1, S_2, \dots, S_k\}$, **such that**

- $|S_1| = |S_k| = 1$.
- For $\forall u \in S_i$ ($1 \leq i < k$), there exists **at least** a $v \in S_{i+1}$ such that $(u, v) \in E$ and $cost(u, v) > 0$.
- For $\forall u \in S_i$ and $v \in S_j$, if $|i - j| > 1$, then $(u, v) \notin E$ and $(v, u) \notin E$.

How to solve it?



- If we know the shortest path from 1 to 5 and 1 to 6, the problem becomes so **easy**!
- Since we can **efficiently** construct the shortest path of **1 to 7** from **1 to 5** and **1 to 6** (just compare two of them).
- We call them as **sub-problems** of 1 to 7.
 - How to **identify** the elements of the subproblems **1 to 5** and **1 to 6**?
 - How to **represent** those subproblems?
 - How to **derive** the relationships with their subproblems?

Solution

- Define that $c(i)$ is the shortest distance from i to e , then

$$c(e) = 0$$

$c(s)$ is the **goal**

$$c(i) = \min_{j \in N(i)} \{c(j) + \text{cost}(i, j)\} \quad (N(i) \text{ is the set of neighbors of } i)$$

- Solve $c(i)$ in reverse order

$$c(7) = 0$$

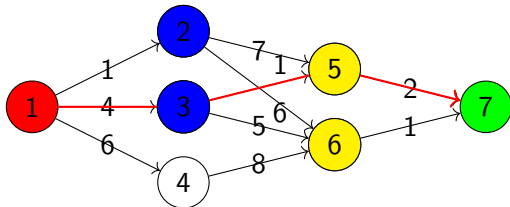
$$c(6) = 1, c(5) = 2$$

$$c(4) = 8 + c(6) = 9$$

$$\begin{aligned} c(3) &= \min\{1 + c(5), 5 + c(6)\} \\ &= \min\{3, 6\} = 3 \end{aligned}$$

$$c(2) = 7, \text{ } c(1) = 7$$

Problem properties



- A complex/large problem is broken simpler/smaller steps/**subproblems**.
- The solution to the larger problem can be constructed **efficiently** from ones of smaller subproblems
- Solutions for overlapped **smaller** subproblems can be **reused** to solve the **larger** problems

A method of solving complex problems by breaking them down into subproblems in recursive manner is called the **Dynamic Programming**.

Formal statements

The term dynamic programming was originally used in the 1940s by Richard Bellman (Bellman equation)

- DP refers specifically to **nesting** smaller **decision** problems inside larger decisions.
- Dynamic was chosen because it sounded **impressive**, **not** because it described how the method worked.
- Programming referred to the use of the method to **find an optimal**.

Dynamic programming is a **mathematical optimization technique** and **algorithmic paradigm** that solves a complex problem by **recursively decomposing** it into simpler, overlapping subproblems, utilizing the **principle of optimal substructure** to combine their solutions into a global optimum. Solutions to subproblems are stored (via **memorization**) to avoid redundant computation, ensuring each subproblem is solved only once.

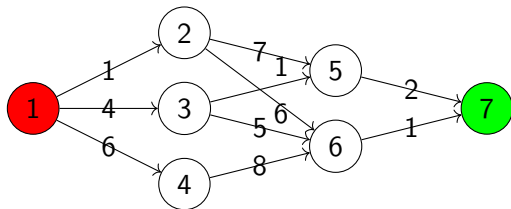
Core Principles

A problem is said to have **Optimal substructure** (Principle of Optimization)

- if its optimal solution can be **constructed** efficiently from optimal solutions to its subproblems, **equally**
- if an optimal solution to the problem **contains within** optimal solutions to its subproblems.
- It indicates subproblems must be only '**slightly**' **smaller** than the larger problem
 - A **constant additive** factor
 - A **multiplicative factor**: D&C Algorithm

A problem said to have **overlapping subproblems** if the problem can be broken down into subproblems that could be **reused** several times.

DP $\approx$ **Recursion** + **Memorization**



1	Identifying subproblems(states)	5-7 or 6-7 ?
2	Validating principle of optimization	What happen if not
3	Defining an optimal value function	Assumed that $c(i)$ is the shortest distance from i to e
4	Deriving the recursive equation	$c(i) = \min_{j \in N(i)} \{ c(j) + \text{cost}(i,j) \}$
5	Solving the recursive equation	$c(7) - c(6) - c(5) - c(4) - c(3) - c(2) - c(1)$
6	Tracebacking the optimal solution	$P(1) - P(3) - P(5) - P(7)$

- 1 What is DP
- 2 0/1 Knapsack Problem
 - Problem statements
 - DP Procedures
 - Solving the recursive equation
- 3 Matrix Multiplication Chains
- 4 All Pairs Shortest Path
- 5 Maximum Non-crossing Subset of Nets
- 6 Longest Common Subsequences

Formal representation:

Given $n > 0$, $c > 0$ and two n -tuples of **positive numbers**:
 $(w_1, w_2, \dots, w_n)$ and $(p_1, p_2, \dots, p_n)$.

we wish to determine the subset
 $T \subset \{1 \dots n\}$ such that

Maximize: $\sum_{i \in T} p_i$

Subjects to : $\sum_{i \in T} w_i \leq c$

If used a n -tuple of indicator variables $(x_1, x_2, \dots, x_n)$, then

Maximize: $\sum_{i=1}^n x_i * p_i$

Subjects to: $\sum_{i=1}^n x_i * w_i \leq c$

Solutions

- Brute force: Try all 2^n possible subsets of T
- Greedy
 - choose items maximizing value ?
 - choose items maximizing value/size
 - the second looks much great, but what if items don't exactly fit (non-divisible items)?
- any others?
 - traceback
 - branch and bound
 - ...
- **Dynamic programming** is one of great choice

Step 1. Identifying subproblems

$$n = 5, c = 10$$

	○	○	○	○	○
p	6	3	5	4	6
w	2	2	6	5	4
x	x_1	x_2	x_3	x_4	x_5

- By fewer items: exclude an item i , what changed?
 - the number of items n becomes $n - 1$, and
 - the capacities become c or $c - w_i$, why?
 - So the subproblem is constrained by **two parameters**.
- By smaller knapsack capacities: reduce the capacity, what changed?
 - the capacities become smaller c'
 - supposed that this is caused by adding an item i ,
 - $c' = c - w_i$,
 - the optimal value: $f(c) = f(c - w_i) + v_i$
 - however, we do not know which i , choose the best

$$f(c) = \max_{i=1, w_i < c}^n \{f(c - w_i) + v_i\}$$

Step 2. Validating Principle of Optimization

Optimal substructure: A problem exhibits optimal substructure if an optimal solution to the problem contains **within it** optimal solutions to its subproblems.

- Supposed that $(x_1, x_2, \dots, x_n)$ is the optimal solution of original problem P^0 .
- Let $(y_1, y_2, \dots, y_{n-1})$ is the optimal solution of its subproblem P' with:
 - items: $1, 2, \dots, n-1$, and
 - capacities: $c - x_n * w_n$.
- We should show that $(y_1, y_2, \dots, y_{n-1}, x_n)$ is no worse than $(x_1, x_2, \dots, x_n)$ to P^0
- How? using a "cut-and-paste" technique (homework)

Step 3. Defining an optimal value function

We have known that a subproblem is constrained by two parameters:

- number of items $n - i - 1$, means that items $i, i + 1, \dots, n$
- capacities c

For the given subproblem, we define its optimal value by $f(i, y)$.

- $f(n, 0) = 0$, and
- $f(n, w_n) = p_n$ if $w_n < c$ else 0
- $f(1, c)$ is **our goal**, why?
- How to get $f(1, c)$ from $f(n, y)$?

Step 4. Deriving the recursive equation

To calculate $f(i, y)$ from $f(i + 1, y)$, we only need know **if item i could be included**

- If $w_i > y$, item i couldn't be included, $f(i, y) = f(i + 1, y)$.
- If $w_i \leq y$, item i might be included
 - if so, $f(i, y) = f(i + 1, y - w_i) + p_i$
 - else, $f(i, y) = f(i + 1, y)$, why?
 - finally, we use the larger one.

$$f(i, y) = \max\{f(i + 1, y), f(i + 1, y - w_i) + p_i\}$$

Step 5. Solve recursive equation

Now we have the recursive equation

$$f(i, y) = \begin{cases} f(i+1, y) & \text{if } 0 \leq y < w_i \\ \max \begin{cases} f(i+1, y - w_i) + p_i \\ f(i+1, y) \end{cases} & \text{if } y \geq w_i \end{cases}$$

and the initial condition

$$f(n, y) = \begin{cases} p_n & \text{if } y \geq w_n \\ 0 & \text{if } 0 \leq y < w_n \end{cases}$$

How to solve them?

- Recursive method
- Non-recursive method
- Tuple method

Recursive method

Algorithm 1: RKnapsack

Input: $n, c, p[1..n], w[1..n]$

Output: the optimal value
 $f(1, c)$

```

1 Function f(int i, int y)
2   if ( $i == n$ ) then
3     return
       ( $y < w[n] ? 0 : p[n]$ );
4   if ( $y < w[i]$ ) then
5     return f( $i+1, y$ );
6   return
     max(f( $i+1, y$ ), f( $i+1, y -$ 
       w[i]) + p[i]);

```

Algorithm complexity:

We use $t(n)$ to denote the algorithm time complexity with n items

$$t(1) = a$$

At the best case(step4)

$$t(n) = t(n-1) + b$$

$$t(n) = \Theta(n)$$

At the worst case(step6)

$$t(n) = 2t(n-1) + b$$

$$t(n) = \Theta(2^n)$$

An example:

$n=5, c=10$

$p=$

$w=$

x



6

2

x_1



3

2

x_2



5

6

x_3



4

5

x_4

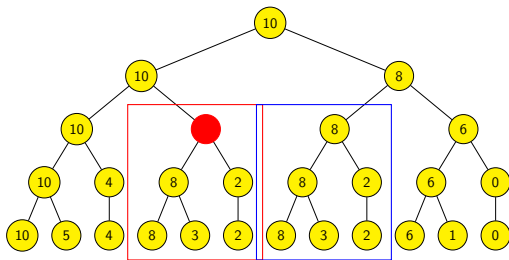


6

4

x_5

The **recursive tree** demonstrates the possibility for **reducing** the complexity!



The number in the node is y value and the layer order corresponds to i , there are total 27 nodes;

The calculations for at least the six nodes could be saved

Memorization: using array

- The algorithm saves all possible values using an array f , so that every $f(i, y)$ is calculated only once.
- It needs $\Theta(nc)$ extra space.
- Its time complexity is $\Theta(nc)$.
 - It is not polynomial: to describe c need $\log_2 c$ bits
 - but pseudo-polynomial: exponential dependence on numerical inputs
- Its disadvantage
 - The capacity c must be an integer
 - The complexity might still be very high when c is large enough, for instance $c = 2^n$

```

1 void knapsack(int *p, int *w, int n, int c,
    int **f)
2 {
3     // initialize f[n][]
4     int yMax = min(w[n] - 1, c);
5     for (int y = 0; y <= yMax; y++)
6         f[n][y] = 0;
7     for (int y = w[n]; y <= c; y++)
8         f[n][y] = p[n];
9     // compute f[i][y], 1 < i < n
10    for (int i = n - 1; i > 1; i--)
11    {
12        yMax = min(w[i] - 1, c);
13        for (int y = 0; y <= yMax; y++)
14            f[i][y] = f[i + 1][y];
15        for (int y = w[i]; y <= c; y++)
16            f[i][y] = max(f[i + 1][y], f[i + 1][y - w[i]] + p[i]);
17    }
18    // compute f[1][c]
19    f[1][c] = f[2][c];
20    if (c >= w[1])
21        f[1][c] = max(f[1][c], f[2][c - w[1]] + p[1]);
22 }

```

Memorization: using a tuple

$n=5, c=10$

$p=$
 $w=$
 x



6
2
 x_1



3
2
 x_2



5
6
 x_3



4
5
 x_4



6
4
 x_5

- Calculate f values using array
 - $f(5, 0) = 0, \dots, f(5, 4) = 6, \dots, f(5, 10) = 6$
 - $f(4, 0) = 0, \dots, f(4, 4) = 6, \dots, f(4, 9) = 10, f(4, 10) = 10$
 - “...” indicate no needs for saving
- Save **step points** only for each i
 - Define a tuple (a, b) , where $a = y$ and $b = f(i, y)$
 - (a, b) corresponds an **optimal loading** value b with capacity a for items $[i..n]$
 - Put **all** tuples into a set P_i
 - P_n can be got easily. $P_n = \{(0, 0), (w_n, p_n)\}$
 - P_1 contains our **goal!** Why?

Tuple method: principle

- Let $Q = \{(s, t) | w_i \leq s < c, (s - w_i, t - p_i) \in P_{i+1}\}$
 - Q corresponds P_i in which item i **has been** selected
 - P_{i+1} corresponds P_i in which item i **has not been** selected
 - thus, $P_i = Q \cup P_{i+1}$
- Merge Q and P_{i+1} to get P_i
 - remove dominated tuples (a tuple (a, b) is dominated by (u, v) if $a > u$, but $b < v$)
 - remove repeated tuples
 - remove tuples with over capacity($a > c$)
- The complexity is still $O(2^n)$.
 - The number of elements in P_i increase exponentially at worst case

Tuple method: an example

$n=5, c=10$

$p=$

$w=$

x



6

2

x_1



3

2

x_2



5

6

x_3



4

5

x_4



6

4

x_5

1. $P(5)=[(0,0), (4,6)]$

$Q=[(5,4), (9,10)]$

2. $P(4)=[(0,0), \mathbf{(4,6)}, (9,10)]$

$Q=[(6,5), (10,11)]$

3. $P(3)=[(0,0), \mathbf{(4,6)}, (9,10), (10,11)]$

$Q=[(2,3), (6,9)]$

4. $P(2)=[(0,0), (2,3), \mathbf{(6,9)}, (9,10), (10,11)]$

$Q=[(2,6), (4,9), \mathbf{(8,15)}, (6,12)]$

5. $P(1)=[(0,0), (2,6), \mathbf{(8,15)}, (6,12)]$

The optimal value is **15** and the optimal solution is **[1,1,0,0,1]** by
tracebacking

- 1 What is DP
- 2 0/1 Knapsack Problem
- 3 Matrix Multiplication Chains**
 - Problem statement
 - DP Procedures
 - Recursive algorithm
 - Iterative algorithm
- 4 All Pairs Shortest Path
- 5 Maximum Non-crossing Subset of Nets
- 6 Longest Common Subsequences

- Production $A \times B$ of two matrices with dimension (m,n) and (n,q) takes $m n q$ multiplications.
- Let's consider three matrices: $A \times B \times C$ with dimension $(100,1)$, $(1,100)$ and $(100,1)$. The product can be done:
 - $((A \times B) \times C)$ takes 20000 multiplications.
 - $(A \times (B \times C))$ takes 200 multiplications.
 - Any other way? No
 - Associative law : $((A \times B) \times C) = (A \times (B \times C))$
 - Commutative law: $A \times B \neq B \times A$
 - the order of multiplications makes a big difference in the final running time!
- **Matrix Multiplication Chains.** Suppose that we multiply q matrices
 - $M(1, q) = M_1 \times M_2 \times \dots \times M_q$
 - There are $q + 1$ $(r_1, r_2, \dots, r_q, r_{q+1})$ numbers(parameters) to constrain their dimensions.

Identifying subproblems

The number of **orders** of multiplications equals to the number of parenthesization:

$$P(q) = \begin{cases} 1 & \text{if } q = 2 \\ \sum_{k=1}^{q-1} p(k)p(q-k) & \text{if } q \geq 2 \end{cases}$$

$$T(q) \approx O(4^q q^{\frac{3}{2}}) \approx O(2^q)$$

In fact, a particular parenthesization can be represented by a binary tree

- The leaves corresponds to matrices
- The root corresponds to the final product
- Interior nodes are intermediate products
- If a tree to be optimal, its subtrees must also be optimal.
- Thus **MPC satisfies the principle of optimization**

Define the optimal function

Define $c(i, j)$ is the **number of multiplication**:

$M_i \times M_{i+1} \times \cdots \times M_j$, We have:

- $c(0, 0) = 0$
- $c(i, i) = 0$
- $c(i, i + 1) = r_i * r_{i+1} * r_{i+2}$
- $c(1, q)$ is **our goal**
- The recursive equation is

$$c(i, j) = \min_{i \leq k < j} \{c(i, k) + c(k + 1, j) + r_i * r_{k+1} * r_{j+1}\}$$

- Let $\text{kay}(i, j)$ is the number minimizing above equation.
 - $\text{kay}(i, i) = 0$
 - $\text{kay}(i, i+1) = i$

An example

Suppose that $q = 5$ and $r = (10, 5, 1, 10, 2, 10)$, find the **optimal** orders of multiplications

	$M(1, 5) = M(1, 2) \times M(3, 5)$	$M(3, 5) = M(3, 4) \times M(5, 5)$
$c(1,5)=\min$	$\{c(1,1)+c(2,5)+500,$ $c(1,2)+c(3,5)+100,$ $c(1,3)+c(4,5)+1000,$ $c(1,4)+c(5,5)+200\}$	$c(1,5)=190, \text{ kay}(1,5)=2$ $c(1,3)=150, \text{ kay}(1,3)=2$ $c(1,4)=90, \text{ kay}(1,4)=2$
$c(2,5)=\min$	$\{c(2,2)+c(3,5)+50,$ $c(2,3)+c(4,5)+500,$ $c(2,4)+c(5,5)+100\}$	$c(2,5)=90, \text{ kay}(2,5)=2$
$c(3,5)=\min$	$\{c(3,3)+c(4,5)+100,$ $c(3,4)+c(5,5)+20\}$ $\min\{300,40\} = 40$	$c(3,5)=40, \text{ kay}(3,5)=4$
$c(2,4)=\min$	$\{c(2,2)+c(3,4)+10,$ $c(2,3)+c(4,4)+100\}$ $\min\{30,150\} = 30$	$c(2,4)=30, \text{ kay}(2,4)=2$

Source codes

```

1  int RC(int i, int j)
2  { // Return c(i,j) and compute kay(i,j)=kay[i][j]
    }.
3  // Avoid recomputations, check if already
    computed
4  if (c[i][j] > 0) return c[i][j];
5  // c[i][j] not computed before, compute now
6  if (i==j) return 0; // one matrix
7  if (i==j-1) { // two matrices
8      kay[i][i+1] = i;
9      c[i][j] = r[i]*r[i+1]*r[i+2];
10     return c[i][j];
11 }
12 // more than two matrices
13 // set u to mini term for k = i
14 int u = RC(i,i) + RC(i+1,j) + r[i]*r[i+1]*r[j
    +1];
15 kay[i][j] = i;
16 // compute remaining min terms and update u
17 for (int k = i+1; k < j; k++) {
18     int t = RC(i,k) + RC(k+1,j) + r[i]*r[k+1]*r
        [j+1];
19     if (t < u) { // smaller min term
20         u = t;
21         kay[i][j] = k; }
22 }
23 c[i][j] = u;
24 return u;
25 }

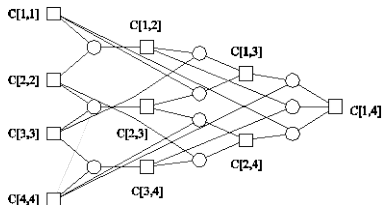
```

```

1  void Traceback(int i, int j, int **kay
    )
2  {
3      if (i == j) return;
4      Traceback(i, kay[i][j], kay);
5      Traceback(kay[i][j]+1, j, kay);
6      cout << "Multiply M " << i << ", "
        << kay[i][j];
7      cout << " and M " << (kay[i][j]+1)
        << ", " << j
        << endl;
8
9  }

```

Revision



The above figure suggest that $c(i, j)$ can be calculated in an iterative way:

$$c(i, i + s) = \min_{i \leq k < i+s} \{c(i, k) + c(k, j) + r_i * r_k * r_{i+s+1}\}$$

$$s = 1, 2, \dots, q$$

Source codes

```
1 void MatrixChain(int r[], int q, int **c, int **kay)
2 { //Compute costs and kay for all Mij's.
3   //initialize c[i][i], c[i][i+1], and kay[i][i+1]
4   for (int i = 1; i < q; i++) {
5     c[i][i] = 0;
6     c[i][i+1] = r[i]*r[i+1]*r[i+2];
7     kay[i][i+1] = i;
8   }
9   c[q][q] = 0;
10  //compute remaining c's and kay's
11  for (int s = 2; s < q; s++)
12    for (int i = 1; i <= q - s; i++) {
13      // min term for k = i
14      c[i][i+s] = c[i][i] + c[i+1][i+s]
15        + r[i]*r[i+1]*r[i+s+1];
16      kay[i][i+s] = i;
17      // remaining mini terms
18      for (int k = i+1; k < i + s; k++) {
19        int t = c[i][k] + c[k+1][i+s]
20          + r[i]*r[k+1]*r[i+s+1];
21        if (t < c[i][i+s]) { // smaller mini term
22          c[i][i+s] = t;
23          kay[i][i+s] = k;
24        }
25      }
26  }
```

- 1 What is DP
- 2 0/1 Knapsack Problem
- 3 Matrix Multiplication Chains
- 4 All Pairs Shortest Path**
 - Problem statement
 - Subproblems identification
 - Iterative algorithm
- 5 Maximum Non-crossing Subset of Nets
- 6 Longest Common Subsequences

What is it?

Input: Given a directed graph $G = (V, E)$ and a matrix (a_{ij})

$$V = \{1, 2, \dots, n\}(\text{nodes})$$

$$W : E \rightarrow R(\text{edge weight function})$$

$$a_{ij} = \begin{cases} w(i, j) & \text{if } (i, j) \in E \\ 0 & \text{if } i = j \\ \infty & \text{otherwise} \end{cases}$$

Output: A $n \times n$ matrix $c(i, j)$ of costs of **shortest paths** for all pairs of vertices.

Assumption: No negative-weight cycles (**why**)

By edges along the path

Define d_{ij}^m the weight of a shortest path from i to j that only uses **at most m** edges

- d_{ij}^{n-1} is **our goal**
- We have known that

$$d_{ij}^0 = \begin{cases} 0 & \text{if } i == j \\ \infty & \text{if } i \neq j \end{cases}$$

$$d_{ij}^1 = \begin{cases} 0 & \text{if } i == j \\ a_{ij} & \text{if } i \neq j \end{cases}$$

Theorem

For $m = 1, 2, \dots, n-1$, we have

$$d_{ij}^m = \min_k \{d_{ik}^{m-1} + a_{kj}\}$$

Proof

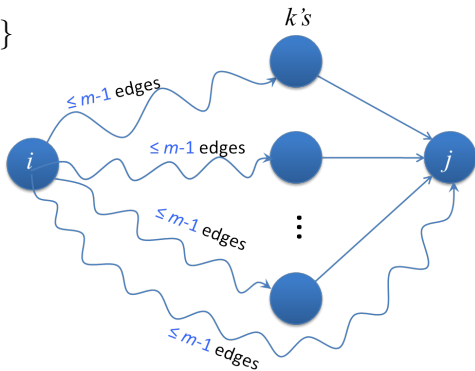
$$d_{ij}^m = \min_k \{d_{ik}^{m-1} + a_{kj}\}$$



for $k=1$ to n

if $d_{ij} > d_{ik} + a_{kj}$

$d_{ij} = d_{ik} + a_{kj}$



Running time $O(n^4)$ - similar to n runs of Bellman-Ford algorithm

The Bellman-Ford algorithm is an algorithm that computes shortest paths from a single source vertex to all of the other vertices in a weighted digraph.

By intermediate vertices along the path

- Define d_{ij}^m weight of a shortest path from i to j that only uses **intermediate vertices** from set $\{1, \dots, m\}$, or
- Define d_{ij}^m weight of a shortest path from i to j that **the orders** of **intermediate vertices** is no larger than **m** .
- d_{ij}^n is our **goal**
- We have known that
 - $d_{ij}^0 = a_{ij}$
 - $d_{ik}^{k-1} = d_{ik}^k$
 - $d_{kj}^{k-1} = d_{kj}^k$

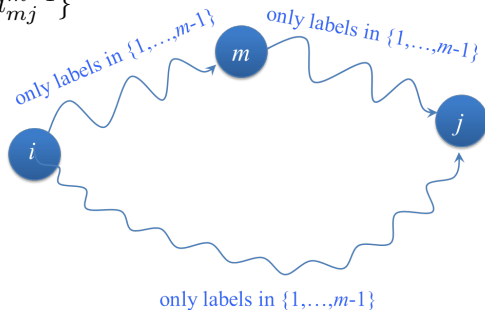
Theorem

For $m = 1, 2, \dots, n-1$, we have

$$d_{ij}^m = \min\{d_{ij}^{m-1}, d_{im}^{m-1} + d_{mj}^{m-1}\}$$

proof

$$d_{ij}^m = \min\{d_{ij}^{m-1}, d_{im}^{m-1} + d_{mj}^{m-1}\}$$



Running time $O(n^3)$ Known as Floyd-Warshall algorithm

Source codes

```

1  template<class T>
2  void AdjacencyWDigraph<T>::AllPairs(T **c, int **kay)
3  {// All pairs shortest paths.
4    // Compute c[i][j] and kay[i][j] for all i and j.
5    // initialize c[i][j] = c(i,j,0)
6    for (int i = 1; i <= n; i++)
7      for (int j = 1; j <= n; j++) {
8        c[i][j] = a[i][j];
9        kay[i][j] = 0;
10     }
11    for (int i = 1; i <= n; i++)
12      c[i][i] = 0;
13
14    // compute c[i][j] = c(i,j,k)
15    for (int k = 1; k <= n; k++)
16      for (int i = 1; i <= n; i++)
17        for (int j = 1; j <= n; j++) {
18          T t1 = c[i][k];
19          T t2 = c[k][j];
20          T t3 = c[i][j];
21          if (t1 != NoEdge && t2 != NoEdge &&
22              (t3 == NoEdge || t1 + t2 < t3)) {
23            c[i][j] = t1 + t2;
24            kay[i][j] = k;}
25        }
26  }

```

Kay[i][j] is used to store the largest vertex in the shortest path from i to j, so that traceback the shortest path

```

1  void outputPath(int **kay, int i, int j)
2  {// Actual code to output i to j path.
3    if (i == j) return;
4    if (kay[i][j] == 0) cout << j << ' ';
5    else {outputPath(kay, i, kay[i][j]);
6          outputPath(kay, kay[i][j], j);}
7  }
8
9  template<class T>
10 void OutputPath(T **c, int **kay, T NoEdge,
11                int i, int j)
12 {// Output shortest path from i to j.
13   if (c[i][j] == NoEdge) {
14     cout << "There is no path from " << i
15         << " to "
16         << j << endl;
17     return;}
18   cout << "The path is" << endl;
19   cout << i << ' ';
20   outputPath(kay,i,j);
21   cout << endl;

```

An example

Adjacent matrix					Distance matrix of shortest path					matrix kay_{ij}				
0	1	4	4	8	0	1	2	3	4	0	0	2	3	4
3	0	1	5	9	3	0	1	2	3	0	0	0	3	4
2	2	0	1	8	2	2	0	1	2	0	0	0	0	4
8	8	9	0	1	5	5	3	0	1	5	5	5	0	0
8	8	2	9	0	4	4	2	3	0	3	3	0	3	0

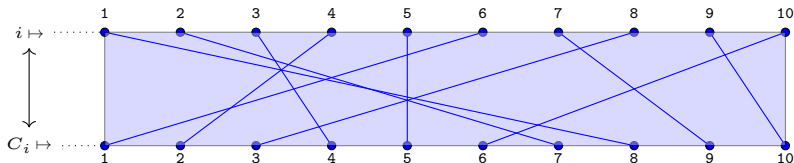
we can traceback the shortest paths from matrix (kay_{ij}) , for example, from 1 to 5

- $kay(1,5)=4$, $kay(4,5)=0$, $4 \rightarrow 5$ is a sub-path
- $kay(1,4)=3$, $kay(3,4)=0$, $3 \rightarrow 4$ is a sub-path
- $kay(1,3)=2$, $kay(2,3)=0$, $2 \rightarrow 3$ is a sub-path
- $kay(1,2)=0$, $1 \rightarrow 2$ is a sub-path
- The final shortest path is $1 \rightarrow 2 \rightarrow 3 \rightarrow 4 \rightarrow 5$

Outline

- 1 What is DP
- 2 0/1 Knapsack Problem
- 3 Matrix Multiplication Chains
- 4 All Pairs Shortest Path
- 5 Maximum Non-crossing Subset of Nets**
 - Problem statement
 - DP solution
- 6 Longest Common Subsequences

What is it?



- A 1-1 map (i, c_i) is called a **subnet**
- Two subnets (i, c_i) and (j, c_j) are **non-crossed** if $i < j$ then $c_i < c_j$
- The set $MNS(i, j) = \{(u, c_u) | u \leq i, c_u \leq j\}$ is called **non-crossing set** if for $\forall(p, c_p)$ and $\forall(q, c_q) \in MNS(i, j)$ then (p, c_p) and (q, c_q) are non-crossed
- Our goal is to find a $MNS(n, n)$ with the **maximum number of elements**
- Define $size(i, j) = |MNS(i, j)|$

Derive the recursive equation

- Our goal is to maximize $\text{size}(n,n)$
- We have known that

$$\text{size}(1, j) = \begin{cases} 0 & \text{if } j < c_1 \\ 1 & \text{if } j \geq c_1 \end{cases}$$

- We want to know the relationship between $\text{size}(i,j)$ and $\text{size}(i-1,j)$
 - if $j < c_i$ then $(i, c_i) \notin \text{MNS}(i-1, j)$, thus $\text{size}(i, j) = \text{size}(i-1, j)$
 - if $j \geq c_i$, there are two cases:
 - put (i, c_i) into $\text{MNS}(i-1, j)$, but (i, c_i) might cross with items in $\text{MNS}(i-1, j)$ or result in smaller size, we have $\text{size}(i, j) = \text{size}(i-1, j)$
 - put (i, c_i) into $\text{MNS}(i-1, j)$, no crossing, then c_{i-1} must be less than $c_i - 1$, else crossed with (i, c_i) , thus we have $\text{size}(i, j) = \text{size}(i-1, c_i - 1) + 1$
 - finally, we choose the maximum of them!

The recursive equation

$$size(i, j) = \begin{cases} size(i-1, j) & \text{if } j < c_i \\ \max\{size(i-1, j), size(i-1, c_i-1) + 1\} & \text{if } j \geq c_i \end{cases}$$

```

1 void mns(int *c, int numberOfNets, int **size)
2 { // Compute size[i][j] for all i and j.
3   // initialize size[1][*]
4   for (int j = 0; j < c[1]; j++)
5     size[1][j] = 0;
6   for (int j = c[1]; j <= numberOfNets; j++)
7     size[1][j] = 1;
8   // compute size[i][*], 1 < i < numberOfNets
9   for (int i = 2; i < numberOfNets; i++)
10  {
11    for (int j = 0; j < c[i]; j++)
12      size[i][j] = size[i-1][j];
13    for (int j = c[i]; j <= numberOfNets; j++)
14      size[i][j] = max(size[i-1][j], size[i-1][c[i]
15        - 1] + 1);
16  }
17  size[numberOfNets][numberOfNets] =
18    max(size[numberOfNets-1][numberOfNets], size[
19      numberOfNets-1][c[numberOfNets]-1]+1);
20 }
```

Outline

- 1 What is DP
- 2 0/1 Knapsack Problem
- 3 Matrix Multiplication Chains
- 4 All Pairs Shortest Path
- 5 Maximum Non-crossing Subset of Nets
- 6 Longest Common Subsequences
 - Problem statement
 - Solutions

Definitions

Subsequence Given a sequence $X = x_1 x_2 \cdots x_m$, another sequence $Z = z_1 z_2 \cdots z_k$ is a subsequence of X if there exists a strictly increasing sequence $i_1 < i_2 < \cdots < i_k$ of indices of X such that for all $j = 1, 2, \dots, k$, we have $x_{i_j} = z_j$.

Example 1: If $X = abcdefg$, $Z = abd$ is a subsequence of X .

Common subsequence Given two sequences X and Y , a sequence Z is a common subsequence of X and Y if Z is a subsequence of **both** X and Y .

Example 2: $X = abcdefg$ and $Y = aaadgfd$. $Z = adf$ is a common subsequence of X and Y .

Longest common subsequence(LCS) A LCS of X and Y is a common subsequence of X and Y with the **longest** length.

- the LCS of two strings may not be unique, for example, strings both acd and abd are LCS of abcd and acbd
- LCS has been successfully applied to many fields
 - Bioinformatics, e.g. long preserved regions in genomes
 - file comparison, e.g. diff
- Solutions
 - Brute force approach: $O(n2^m)$
 - DP approach: $O(nm)$

Derive recursive equation

- Let's consider the prefixes of X and Y
 - $X[1..i]$ i -th prefix of $X[1..m]$
 - $Y[1..j]$ j -th prefix of $Y[1..n]$
- Subproblem: define $c[i,j] = |LCS(X[1..i], Y[1..j])|$
 - $c[m,n]$ is **our goal**
 - initialization: $c[1,1]=1$ if $x_1 = y_1$ otherwise 0
- To derive recursive relationship, we use the following theorem

Theorem: Let $X=x_1x_2\cdots x_m$ and $Y=y_1y_2\cdots y_n$ be two sequences, and $Z=z_1z_2\cdots z_k$ be any LCS of X and Y , then

- ① If $x_m = y_n$, then $z_k = x_m = y_n$ and $Z[1..k-1]$ is an LCS of $X[1..m-1]$ and $Y[1..n-1]$.
- ② If $x_m \neq y_n$, then $z_k \neq x_m$ implies that Z is an LCS of $X[1..m-1]$ and Y .
- ③ If $x_m \neq y_n$, then $z_k \neq y_n$ implies that Z is an LCS of X and $Y[1..n-1]$.

Recursive equation and implementation

$$c[i, j] = \begin{cases} 0 & \text{if } i=0 \text{ or } j=0 \\ c[i-1, j-1] + 1 & \text{if } x[i]=y[j] \\ \max\{c[i-1, j], c[i, j-1]\} & \text{otherwise} \end{cases}$$

```

1  LCS(X,Y,m,n,b)
2  for i=1 to m do
3      c[i,0]=0;
4  for j=0 to n do
5      c[0,j]=0;
6  for i=1 to m do
7      for j=1 to n do
8          { //b[i,j] stores the directions.
9            if x[i] == y[j] then
10               c[i,j]=c[i-1,j-1]+1;
11               b[i,j]=1; //1-diagonal,
12            else if c[i-1,j]>c[i,j-1] then
13               c[i,j]=c[i-1,j]
14               b[i,j]=2; //2-up,
15            else c[i,j]=c[i,j-1]
16               b[i,j]=3; //3-
                      forward.
17      }
```

```

1  PrintLCS(b,X,i,j)
2  i=m
3  j=n;
4  if i==0 or j==0 then exit;
5  if b[i,j]==1 then
6  {
7      i=i-1;
8      j=j-1;
9      print x[i];
10 }
11 if b[i,j]==2 i=i-1
12 if b[i,j]==3 j=j-1
13 Goto Step 3.
```


Coming up: Backtracking
Algorithm